PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING

SUBJECT: SOFTWARE ARCHITECTURE

PROJECT FINAL REPORT: MOVIE TICKET BOOKING SYSTEM

Instructor :	Vu Quang Dung
Student Name:	Pham Xuan Bach
Student ID:	23010283
Email:	23010283@st.phenikaa-uni.edu.vn
Course class:	CSE703110-1-2-25(N01)

Ha Noi, February 2026

Contentsn

1. EXECUTIVE SUMMARY

1.1 Goal:

The primary objective of this project was to refactor a legacy Layered Monolith movie ticket booking application into a distributed Microservices Architecture.

Architectural Pattern: The system adopts a Microservices style utilizing Database-per-Service isolation and a Hybrid Communication Pattern (Synchronous REST APIs for reads and Asynchronous AMQP via RabbitMQ for writes).

1.2 Final Achievement:

The team successfully decomposed the application into independent containers (Gateway, Movie, Booking, Payment, Notification). The final system is orchestrated via Docker Compose, demonstrating high scalability, fault tolerance, and a clear separation of concerns, successfully addressing the limitations of the previous monolithic version.

2. PROJECT REQUIREMENTS & GOALS

2.1 Functional Requirements (Core Features)

- Movie Management (CRUD): Admin capabilities to Create, Read, Update, and Delete movies and showtimes.
- Booking Transaction: A complex workflow allowing users to select seats, lock them, and process payments.
- Real-Time Messaging: Event-driven notifications (Email/Tickets) and Invoice generation upon successful booking.
- Search & Discovery: High-performance movie searching for end-users.

2.2 Quality Attributes (Targeted Goals)

- Scalability: The ability to scale specific services (e.g., Booking Service) independently during high-traffic events.
- Availability & Fault Tolerance: The system must continue to accept bookings even if the Notification or Payment services are temporarily down (achieved via Message Queues).
- Performance:
    - *Read Performance:* Fast response times for movie listings using direct API calls.

- *Write Performance:* Non-blocking booking experience using asynchronous processing.

3. ARCHITECTURAL DESIGN & IMPLEMENTATION

3.1 Technical Stack

- Backend: Python 3.13 (Flask) for lightweight microservices.
- Frontend: Vanilla JavaScript (ES6+) and Bootstrap 5, bundled with Vite.
- Database: SQLite (Strict Database-per-Service implementation).
- Message Broker: RabbitMQ (Pika client) for event-driven communication.
- Infrastructure: Docker.

3.2 Implementation of Core Logic (Operational Phases)

Phase A: Infrastructure Bootstrap (Containerization)

- Orchestration: We use docker-compose.yml as the infrastructure blueprint. An internal Bridge Network is established to allow services to communicate via Hostnames (Service Discovery) rather than static IPs.
- Idempotent Initialization: Upon startup, each service checks for its specific database file (movies.db, booking.db). If absent, it automatically executes SQL schemas to create tables and seed initial data. This ensures strict data isolation with no cross-service foreign keys.

Phase B: Synchronous Operations (Read-Heavy Flows)

- API Gateway Pattern: The Flask Gateway (Port 5000) acts as the Single Entry Point. It intercepts Frontend requests, validates UUID-based Security Tokens stored in the database, and routes traffic to internal services (e.g., routing /api/movies to movie-service:5001).
- Direct Querying: For information retrieval, the Gateway forwards requests to the Movie Service, which queries its local DB and returns JSON directly. This minimizes latency for UI rendering.

Phase C: Asynchronous Operations (Write-Heavy Flows)

- Event-Driven Booking:
    1. Atomic Transaction: The Booking Service locks the seat in its local DB.
    2. Producer: Immediately upon success, it publishes an *OrderPlacedEvent* directly to the *ticket_events* queue.

3. Non-Blocking Response: The user receives a "Success" HTTP 201 response immediately, without waiting for email processing.

- Consumer Workers: The Notification and Payment services run as background workers. They consume messages from the queue to generate invoices and send emails via SMTP. If a worker crashes, the message remains in the queue (Persistence), ensuring no data loss.

Phase D: Technical Optimization

- Frontend Debounce: To prevent API overload, we implemented a custom useDebounce hook in React. This limits search requests by only firing after the user stops typing for 500ms, reducing Gateway load by approximately 80%.

4. USE CASE SPECIFICATIONS

4.1 System & Authentication (UC-01 to UC-03)

UC-01: API Gateway Routing & Verification: Handles request routing, validates API Keys, and verifies User Tokens.

ID and Name:	UC-01: API Gateway Routing & Verification
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	The Gateway acts as the single entry point (Port 5000). It intercepts all requests, validates the API Key, verifies User Tokens (if present), and routes the request to the appropriate microservice.
Trigger:	Any HTTP Request sent to *http://localhost:5000*
Preconditions:	PRE-1: Gateway Service is running on Port 5000. PRE-2: User Service is running to verify tokens.
Postconditions:	POS-1: Request is successfully forwarded to the downstream service. POS-2: User identity headers (X-User-Email, X-User-Role) are injected.
Normal flow:	1. Client sends a request with Header peko-key. 2. Gateway validates peko-key. 3. Gateway checks for Authorization: Bearer <token> header.

	4. Gateway calls User Service to check token existence in *tokens* table. 5. User Service returns user profile and role. 6. Gateway injects user info into headers. 7. Gateway forwards request to the target service (Movie/Booking/Payment). 8. Gateway returns the response from the target service to the client.
Alternative Flows:	4a. Invalid Token: If User Service returns 401, Gateway denies request immediately.
Exceptions:	2a. Invalid API Key: Returns 401 Unauthorized. 5a. User Service Down: Returns 500 Internal Server Error.
Priority:	High

UC-02: User Registration: Allows new users to create an account.

ID and Name:	UC-02: User Registration
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	Allows a new user to create an account in the system.
Trigger:	Guest sends: *POST /api/auth/register*
Preconditions:	PRE-1: Email must not already exist in users.db.
Postconditions:	POS-1: New user record created with role 'user'.
Normal flow:	1. User submits email and password. 2. Gateway forwards to User Service. 3. User Service hashes the password. 4. User Service saves the record to users.db. 5. System returns success message.

Alternative Flows:	none
Exceptions:	4a. Email Exists: Returns 400 Bad Requests.
Priority:	High

UC-03: User Login: Authenticates users and issues Access Tokens (JWT).

ID and Name:	UC-03: User Login
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	Authenticates a user and issues a *Database-backed UUID Token* for accessingaccessing protected resources.
Trigger:	User sends *POST /api/auth/login*
Preconditions:	PRE-1: User/admin account must exist.
Postconditions:	POS-1: System returns a valid Access Token. POS-2: Client stores this token for future requests.
Normal flow:	1. User submits email and password. 2. Gateway forwards request to User Service. 3. User Service verifies email exists. 4. User Service validates password hash. 5. User Service generates a random *UUID Token*, stores it in the *tokens* table, and returns it. 6. System returns the Token to the user.
Alternative Flows:	none
Exceptions:	3a. User Not Found / Wrong Password: Returns 401 Unauthorized.

Priority:	High

4.2 Movie & Showtime Management (Admin) (UC-04 to UC-09)

UC-04: Create Movie: Adds a new movie to the system.

ID and Name:	UC-04: Create Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Adds a new movie title to the system database.
Trigger:	Admin sends POST /api/movies
Preconditions:	PRE-1: User has 'admin' role. PRE-2: Movie ID must be unique.
Postconditions:	POS-1: New movie record is inserted into movies.db.
Normal flow:	1. Admin submits movie details (ID, Title, Genre, Duration). 2. Gateway checks Admin Token. 3. Movie Service checks if ID already exists. 4. Movie Service inserts data into movies table. 5. System returns "Movie Created" message.
Alternative Flows:	none
Exceptions:	3a. Duplicate ID: Returns 400 Bad Request.
Priority:	High

UC-05: Update Movie: Updates existing movie information.

ID and Name:	UC-05: Update Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin

Description:	Allows Admin to correct or update movie information (Title, Genre, Duration).
Trigger:	Admin sends PUT /api/movies/{id}
Preconditions:	PRE-1: Movie exists. PRE-2: User has 'admin' role.
Postconditions:	POS-1: Movie details updated in movies.db.
Normal flow:	1. Admin submits new details (e.g., fix typo in Title). 2. Gateway verifies Admin Token. 3. Movie Service updates the record in database. 4. System returns success message.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-06: Cascade Delete Movie: Deletes a movie along with its showtimes and triggers automatic ticket refunds.

ID and Name:	UC-06: Cascade Delete Movie
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Deletes a movie and automatically deletes all associated showtimes and refunds all booked tickets.
Trigger:	Admin sends DELETE /api/movies/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist.
Postconditions:	POS-1: Movie, Showtimes, and Bookings are deleted.

	POS-2: REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin requests deletion. 2. Movie Service finds all associated showtimes. 3. Loop: Trigger Refund Process for each showtime. 4. Delete all showtimes. 5. Delete the movie record. 6. Returns the count of refunded tickets.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-07: Create Showtime: Schedules a movie and automatically generates available seats.

ID and Name:	UC-07: Create Showtime
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Schedule a movie and generate seats.
Trigger:	Admin sends POST /api/showtimes
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Movie ID must exist. PRE-3: Time is valid. PRE-4: The total number of seats must not exceed 64.
Postconditions:	POS-1: Showtime saved in movies.db. POS-2: Seats created in booking.db.

Normal flow:	1. Admin sends time and price. 2. Movie Service saves showtime. 3. Movie Service calls Booking Service (/internal/seats) to generate seats. 4. Returns success.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-08: Update Showtime & Auto-Refund: Updates showtime details (triggers refunds if the price changes).

ID and Name:	UC-08: Update Showtime & Auto-Refund
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Update showtime. If price changes, existing bookings are automatically cancelled and refunded.
Trigger:	Admin sends PUT /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Update successful. POS-2: (If price changed) Tickets cancelled, REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin changes price. 2. Movie Service detects price change.

	3. Calls Booking Service to cancel all tickets. 4. Publishes REFUND_NOTIFICATION to RabbitMQ. 5. Updates new price in DB.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-09: Delete Showtime: Deletes a specific showtime and refunds any sold tickets.

ID and Name:	UC-09: Delete Showtime
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	Admin
Description:	Delete a specific showtime and refund sold tickets.
Trigger:	Admin sends DELETE /api/showtimes/{id}
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: The Showtime ID must exist.
Postconditions:	POS-1: Showtime deleted. POS-2:REFUND_NOTIFICATION is sent to the notification_events queue.
Normal flow:	1. Admin requests deletion. 2. Gateway verifies Admin Role. 3. Trigger Refund Process. 4. Delete record in DB. 5. Returns success message.

Alternative Flows:	none
Exceptions:	none
Priority:	High

4.3 Browsing & Booking (User) (UC-10 to UC-15)

UC-10: Search Movies: Lists all movies or searches for specific ones by title/ID.

ID and Name:	UC-10: Search Movies
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	List all movies or search by title or show one movies by id
Trigger:	GET/api/movies(with?query=...) GET/api/movies/{id}
Preconditions:	PRE-1: Movie Service is running. PRE-2: Database has movie records (optional).
Postconditions:	POS-1: Returns list of movies matching query. POS-2: No database modification.
Normal flow:	1. User sends query. 2. Service performs LIKE search. 3. Returns list.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

UC-11: View Showtimes Schedule: Displays available showtimes and pricing.

ID and Name:	UC-11: View Showtimes Schedule
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User, Admin
Description:	View list of available showtimes and prices.
Trigger:	GET /api/showtimes
Preconditions:	PRE-1: Movie Service is running.
Postconditions:	POS-1: Returns list of showtimes. POS-2: No database modification.
Normal flow:	1. User requests schedule. 2. System returns list of showtimes.
Alternative Flows:	none
Exceptions:	none
Priority:	High

UC-12: Book Ticket: Reserves a seat and publishes an order event to the system.

ID and Name:	UC-12: Book Ticket
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	User reserves a seat. Price is fetched dynamically.
Trigger:	POST /api/bookings
Preconditions:	PRE-1: User must be logged in. PRE-2: Seat must be 'available'.
Postconditions:	POS-1: Booking created.

	POS-2: The OrderPlacedEvent event is sent to the ticket_events queue.
Normal flow:	1. User selects seat. 2. Service checks availability. 3. Calls Movie Service to fetch price. 4. Creates booking and updates seat status. 5. Sends event to RabbitMQ.
Alternative Flows:	none
Exceptions:	2a. Seat Taken: Returns 400 Bad Request.
Priority:	High

UC-13: View Bookings: Allows users to view their personal booking history.

ID and Name:	UC-13: View Bookings
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	View history of bookings made by the user.
Trigger:	GET /api/bookings
Preconditions:	PRE-1: User must be logged in.
Postconditions:	POS-1: Returns list of bookings for that user.
Normal flow:	1. Gateway extracts email from Token. 2. Service filters bookings by email. 3. Returns list.
Alternative Flows:	none
Exceptions:	none

Priority:	High

UC-14: View Booking Detail: Shows specific details for a single ticket.

ID and Name:	UC-14: View Booking Detail
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	View specific details of a single ticket.
Trigger:	GET /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in. PRE-2: User must own the ticket.
Postconditions:	POS-1: Returns ticket details.
Normal flow:	1. User requests ID. 2. System verifies ownership. 3. Returns details.
Alternative Flows:	none
Exceptions:	2a. Not Owner: Returns 403 Forbidden.
Priority:	High

UC-15: Cancel Booking (Manual): Allows users to manually cancel their own bookings.

ID and Name:	UC-15: Cancel Booking (Manual)
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	User cancels their own ticket. Note: Manual cancellation does NOT trigger a refund event in the current implementation.
Trigger:	DELETE /api/bookings/{id}
Preconditions:	PRE-1: User must be logged in.

	PRE-2: User must own the ticket.
Postconditions:	POS-1: Booking deleted. POS-2: Seat becomes available.
Normal flow:	1. User requests cancellation. 2. System verifies ownership. 3. Deletes booking. 4. Frees up seat.
Alternative Flows:	none
Exceptions:	none
Priority:	Medium

4.4 Payment (UC-16)

UC-16: Process Payment: Processes mock transactions, logs payments, and generates invoices.

ID and Name:	UC-16: Process Payment
Create by:	Nguyen Gia Bao / Date created: 18/01/2026
Primary Actor:	User
Description:	Pay for a ticket and generate an invoice.
Trigger:	POST /api/payments
Preconditions:	PRE-1: User must have the 'admin' role. PRE-2: Valid Booking ID exists.
Postconditions:	POS-1: Payment logged. POS-2: INVOICE_PRINT event sent.

Normal flow:	1. User submits Booking ID. 2. Gateway verifies Token and routes to Payment Service. 3. Payment Service calls Booking Service (Internal API) to verify the booking exists and get the correct amount. 4. Payment Service processes the mock transaction. 5. Payment Service publishes InvoiceGenerated event to RabbitMQ. 6. System returns "Payment Successful" message to User.
Alternative Flows:	none
Exceptions:	3a. Booking Not Found: Payment Service returns 404. 3b. Already Paid: Payment Service returns 400 Bad Request.
Priority:	High

5. COMPLETED TASK DETAILS

5.1 Deployment Architecture Overview

The project is designed using a Microservices architecture, with the entire system containerized using Docker and orchestrated via Docker Compose. The system consists of 7 independent backend services, 1 frontend service, 1 message broker (RabbitMQ), and 1 seeder service.

5.2 Docker Configuration (Dockerfile Strategy)

Instead of maintaining multiple separate Dockerfiles, the project adopts a Shared Dockerfile strategy (***Dockerfile.backend***) to optimize build time and layer caching:

- Base Image: ***python:3.9-slim*** (lightweight and production-optimized).
- Working Directory: ***/app***.
- Environment Variables:
  - ***PYTHONPATH=/app/src*** to ensure correct module imports.

- ***PYTHONUNBUFFERED=1*** so logs are flushed immediately to the console (critical for debugging in Docker).
- Dependency Management: Installs libraries from a shared ***requirements.txt***.

5.3 Microservices Configuration Details

The system is defined in ***docker-compose.yml*** with a well-structured dependency graph.

5.3.1 Core Infrastructure

- API Gateway (Port 5000):
  - Acts as the single entry point, routing requests to downstream services (Movie, Booking, Payment, User).
  - Has network access to all backend services.
- RabbitMQ (Message Broker – Ports 5672 / 15672):
  - Uses the ***rabbitmq:3-management*** image to provide a web-based management UI.
  - Health Check: Executes ***rabbitmq-diagnostics -q*** ping every 30 seconds to ensure readiness before backend services start (dependency condition: ***service_healthy***).

5.3.2 Functional Backend Services

Services communicate through the internal Docker network using DNS-based service discovery, for example:
http://payment_service:5003

Service	Port	Primary Responsibility	Dependencies
Movie Service	5001	Manage movies & showtimes	RabbitMQ
Booking Service	5002	Handle bookings & seat locking	RabbitMQ, Movie Service
Payment Service	5003	Process payments, store cards	RabbitMQ, Booking Service
User Service	5004	Manage user authentication	RabbitMQ

5.3.3 Consumer Worker

Notification Service (No Port):

- A pure worker service that does not expose any HTTP port.
- Responsibility: Continuously consumes queues from RabbitMQ to send emails or print invoices.

5.3.4 Client & Utilities

- Frontend Service (Port 5173): A React application (Vite) that connects directly to the API Gateway.
- DB Seeder: A one-off container used to initialize sample data for the User and Movie services.

5.4 Event-Driven Architecture with RabbitMQ

The system adopts asynchronous communication to improve decoupling and reliability.

5.4.1 Persistence Mechanisms (Data Durability)

- Durable Queues: Three main queues (ticket_events, invoice_events, notification_events) are declared with durable=True.
- Persistent Messages: All messages are published with delivery_mode=2, ensuring no data loss even if RabbitMQ restarts.

5.4.2 Event Processing Flow

The Notification Service acts as a multi-purpose consumer, handling four main event types:

OrderPlacedEvent: Sends ticket confirmation emails after successful booking.

INVOICE_PRINT: Generates invoices after successful payment (from the Payment Service).

SHOWTIME_CHANGED: Notifies customers when showtime schedules are updated.

REFUND_NOTIFICATION: Sends refund notifications when a showtime is canceled.

5.4.3 Fault Tolerance Mechanisms

- Connection Retry Logic: The worker automatically retries connections (retry interval: 5 seconds) if RabbitMQ is not ready or if the connection is unexpectedly lost.
- Graceful Shutdown: Handles interrupt signals (KeyboardInterrupt) to safely close connections and prevent data loss during in-progress processing.

5.5 Payment Service Details

The Payment Service clearly demonstrates the coordination between REST APIs and event-driven processing.

5.5.1 REST API

- Provides CRUD endpoints for payment methods (***/api/payment-methods***), ensuring security by returning only the last four digits of card numbers (card masking).

- The ***/api/payments*** endpoint executes a complex workflow: booking validation → status update → invoice event publishing.

5.5.2 Event Integration

Immediately after a successful payment, the service does not print the invoice directly (which would increase response time for the user). Instead, it publishes an **INVOICE_PRINT** event to the ***invoice_events*** queue, significantly reducing latency and improving the end-user experience.